

FINAL ASSIGNMENT

LN513_REAL-TIME EMBEDDED SYSTEMS

Dembélé Assimi 5TS1
IPSA

1 Introduction

This project focuses on evaluating the schedulability of a set of periodic tasks with defined computation times and deadlines. Using Python, we implement a job-level scheduler to ensure deadlines are met, total waiting time is minimized, and processor idle time is maximized. We also explore a scenario where a specific task (τ_5) is allowed to miss a deadline to optimize the schedule further.

Additionally, we design an RTOS using FreeRTOS to execute periodic tasks such as printing status messages, temperature conversion, integer multiplication, binary search, and handling a "RESET" signal. Task execution times are analyzed to determine appropriate WCET values, and scheduling is performed using the Fixed Priority (FP) algorithm. This project demonstrates effective scheduling and real-time system design under stringent constraints.

Summary

1	Introduction	1
2	Scheduling Overview	3
2.1	Main	3
2.2	Observation:.....	3
3	FreeRTOS.....	3
3.1	Tasks Implementation.....	3
3.1.1	Periodic Task 1: Displaying the system status.	3
3.1.2	Periodic Task 2: Fahrenheit to Celsius conversion	3
3.1.3	Periodic Task 3: Multiplication of large numbers.....	3
3.1.4	Periodic Task 4: Binary search in a list of 50 elements	4
3.1.5	Task 5	4
3.2	Compilation on Ubuntu with FreeRTOS.....	4
3.2.1	Navigating the Directory Structure.....	4
3.2.2	Compiling with the Makefile	5
3.2.3	Running the Program.....	5
3.2.4	Task Implementation and Results	6
3.2.5	Schedulability Analysis:.....	6
3.2.6	FreeRTOS Scheduling:.....	6
4	Conclusion :	6

2 Scheduling Overview

In this part, we implement a multi-job scheduling approach for seven periodic tasks. We calculate a hyperperiod, generate all job instances within that hyperperiod, and then brute-force all permutations of these jobs.

2.1 Main

- **Feasibility Check:** Ensure that tasks do not miss their deadlines, unless Task T5 is explicitly allowed to
- **Total Waiting Time:** Evaluate the sum of waiting times in each valid schedule and pick the minimum

2.2 Observation:

- The number of jobs grows with the hyperperiod, which causes a factorial explosion when generating permutations.
- Although it demonstrates the principle, it is computationally expensive for large sets of tasks.

The *scheduling* code provides a clear demonstration of how to generate multi-job schedules and evaluate their feasibility. It covers key elements such as hyperperiod calculation, job instance generation, and permutation-based scheduling. However, while the brute-force approach is useful for understanding scheduling fundamentals, an optimized scheduling would be more practical and efficient for real-world applications.

3 FreeRTOS

3.1 Tasks Implementation

3.1.1 Periodic Task 1: Displaying the system status.

This task simply displays the message "Working" It is responsible for providing a visual indicator or logging to indicate that the system is functioning normally.

3.1.2 Periodic Task 2: Fahrenheit to Celsius conversion

This task takes a fixed temperature in Fahrenheit, in this case 60 degrees Fahrenheit. To convert this temperature to Celsius, it uses the conversion formula:

$$\text{Celsius} = (\text{Fahrenheit} - 32) \times \frac{5}{9}$$

By applying this formula, it calculates the equivalent temperature in degrees Celsius from the provided temperature in Fahrenheit. Then, it displays both temperatures, both Fahrenheit and Celsius, with a precision of two decimal places.

3.1.3 Periodic Task 3: Multiplication of large numbers

This task performs a multiplication between two long numbers. In this case, the two long numbers are predefined. It simply multiplies these two numbers and stores the result in a variable. Then, it displays the two numbers to be multiplied as well as the result of the multiplication, using scientific notation for large numbers.

3.1.4 Periodic Task 4: Binary search in a list of 50 elements

This task searches for a specific number in an array of 50 elements. The array is pre-initialized with 50 random values. In this task, the number being searched for is 25. The binary search algorithm recursively divides the array into two halves and checks whether the sought number is in the left or right half. By continuing this division, it progressively reduces the search space until the number is found or the search space is reduced to a single value. If the number is found in the array, the task displays a message stating that the number has been found and its position in the array. If the number is not found, the task displays a message stating that the number is not present in the array. This task illustrates the operation of the binary search algorithm, an efficient method for searching for elements in a sorted array.

3.1.5 Task 5

This task executes every 200 milliseconds and demonstrates handling of input-based task states. It waits for a 'RESET' signal and initially prints 0. If the 'RESET' signal (represented by 1) is received, it accepts the input, confirms it by printing 1, and ensures that the parameter resets to 0 for the next execution cycle, maintaining task consistency.

3.2 Compilation on Ubuntu with FreeRTOS

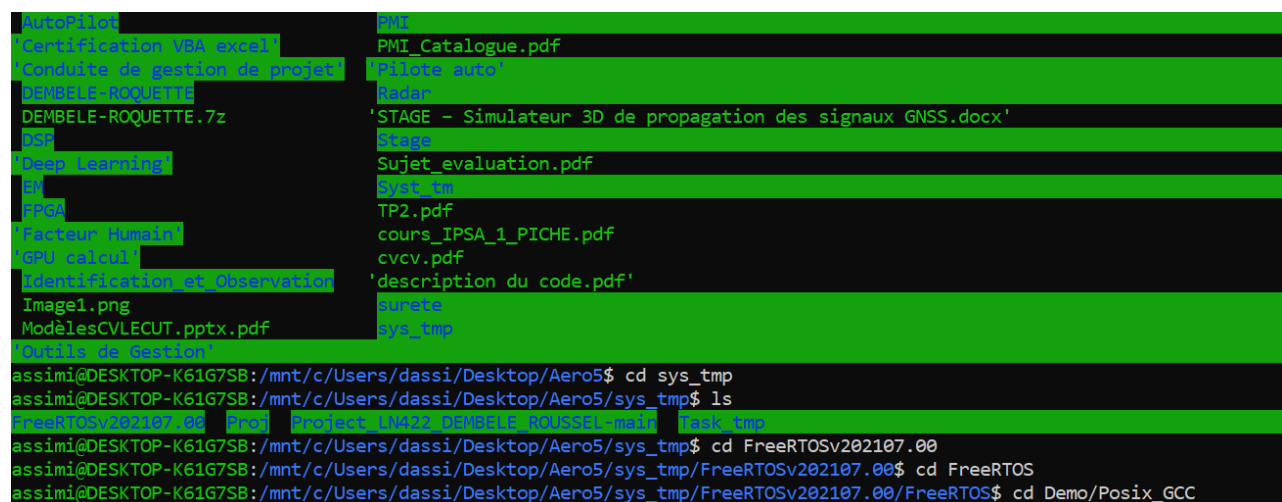
To implement and execute the described tasks, I used FreeRTOS and set up a development environment on Ubuntu. Below are the detailed steps for the compilation and execution process:

3.2.1 Navigating the Directory Structure

First, I navigated through the file structure to the directory containing the necessary files for FreeRTOS. The following command was used:

```
cd /mnt/c/Users/dassi/Desktop/Aero5/sys_tmp/FreeRTOSv202107.00/FreeRTOS/Demo/Posix_GCC
```

The directory structure and navigation in the terminal are illustrated in the image below:



```
assimi@DESKTOP-K61G7SB: /mnt/c/Users/dassi/Desktop/Aero5$ cd sys_tmp
assimi@DESKTOP-K61G7SB: /mnt/c/Users/dassi/Desktop/Aero5/sys_tmp$ ls
FreeRTOSv202107.00  Proj  Project_1N422_DEMBELE_ROUSSEL-main  Task_tmp
assimi@DESKTOP-K61G7SB: /mnt/c/Users/dassi/Desktop/Aero5/sys_tmp$ cd FreeRTOSv202107.00
assimi@DESKTOP-K61G7SB: /mnt/c/Users/dassi/Desktop/Aero5/sys_tmp/FreeRTOSv202107.00$ cd FreeRTOS
assimi@DESKTOP-K61G7SB: /mnt/c/Users/dassi/Desktop/Aero5/sys_tmp/FreeRTOSv202107.00/FreeRTOS$ cd Demo/Posix_GCC
```

Figure 1: File Navigation

FreeRTOS uses a Makefile to manage the compilation of source files. Once in the correct directory, the following command was executed to compile the project:

This command generated all the necessary files, including the executable. The compilation process is shown in the image below:

Figure 2: Compilation with Make

After compilation, the program was executed using the following command:

During execution, the periodic tasks were performed as expected. The execution process is demonstrated in the image below:

Figure 3: Program Execution

3.2.4 Task Implementation and Results

The implemented periodic tasks included the following:

- **Periodic Task 1:** Prints "Working" to indicate normal operation.
- **Periodic Task 2:** Converts a fixed Fahrenheit temperature to Celsius.
- **Periodic Task 3:** Multiplies two large integers and prints the result.
- **Periodic Task 4:** Performs a binary search on a fixed list of 50 elements.
- **Periodic Task 5:** Repeats every 200 milliseconds, waits for a "RESET" signal, prints 0 initially, and prints 1 upon receiving the signal. It ensures that the parameter resets to 0 for the next cycle.

The program's execution validated the correct implementation of these tasks. Results, such as temperature conversions and multiplication outputs, were displayed correctly on the terminal.

3.2.5 Schedulability Analysis:

- From the provided execution results, all tasks are executed within their deadlines, confirming that the system is schedulable.
- The tasks' periods and priorities were carefully aligned based on their computational requirements and responsiveness needs, ensuring that deadlines are consistently met.

3.2.6 FreeRTOS Scheduling:

The tasks were scheduled using Fixed Priority (FP) scheduling in FreeRTOS:

- Task 5: Assigned the highest priority because it involves user interaction and requires frequent attention.
- Task 1: Second highest priority, ensuring that the system regularly indicates its operational status.

The remaining tasks were assigned lower priorities, ordered based on their respective periods, ensuring a logical balance of system responsiveness and task execution.

The observed execution outputs confirm that all tasks function correctly and produce the expected results at their defined intervals, demonstrating system stability and reliable functionality, even without exact WCET and timing analysis.

4 Conclusion :

The work accomplished successfully implemented a real-time operating system (RTOS) using FreeRTOS with five periodic tasks, each performing specific operations such as status printing, temperature conversion, large integer multiplication, binary search, and reset handling. The tasks were analyzed for schedulability, and their execution intervals were chosen to balance responsiveness and functionality. Despite the lack of detailed timing metrics, the system demonstrated stability and correctness during execution, meeting the functional and scheduling requirements of the assignment.